Prometheus

Introduction

What is Prometheus ?

Prometheus is a monitoring system that collects metrics and stores them in a time series database.

What is a time-series database?

A time-series database refers to the recording of changes over time.

What are metrics ?

Metrics are numbers that track changes over time.
What you measure depends on your app — a web server might track response times, while a database might track active connections.

Imagine your pizza delivery app slows down on Super Bowl Sunday. You check your metrics — like requests per second — and see it jumped from 100 to 1,000. This is your time series: that number recorded over time, showing exactly when the spike happened. You add more servers, the numbers drop, problem solved.

Why use Prometheus ?

Collecting metrics in a time-series database is useful for many matters like:

- Enabling metric correlations over time.
- Unveiling patterns and trends.
- Defining meaningful thresholds based on proven alerts.
- Providing feedback on enforced policies or SLAs.

Metrics

Labels

Each metric gets labels attached to it (like `method="POST"`, `endpoint="/api/tracks"`), and you can then query across those labels to slice and filter your data however you need.

Notations

a time series with the metric name api_http_requests_total and the labels `method="POST"` and `handler="/messages"` could be written like this:
`api_http_requests_total{method="POST", handler="/messages"}`
This is the same notation that OpenTSDB uses.

Metrics Types

Counter

A counter only ever goes up (or resets on restart). Use it for things that accumulate over time like requests or errors. Never use it for values that can go down — use a gauge for that instead.

Gauge

A gauge can go up and down freely. Use it for current state values like memory usage, temperature, or active connections.

Histogram

A histogram samples observations (like request durations or response sizes) and counts them in configurable buckets.
It exposes 3 time series automatically:

- `<basename>_bucket{le="<bound>"}` (cumulative bucket counters)
- `<basename>_sum` (sum of all values)
- `<basename>_count` (total count of observations).

Use `histogram_quantile()` to calculate quantiles, and note that buckets are cumulative.

From v2.40, native histograms are experimentally supported — they use a single time series with dynamic buckets, offering higher resolution at lower cost.

Summary

Similar to a histogram, a summary samples observations (like request durations) and also provides `_sum` and `_count`.
The key difference is that it calculates quantiles (ϕ) directly on the client over a sliding time window, exposing them as `<basename>{quantile="< $\phi$ >"}`. See histograms and summaries for when to use one over the other.

Summary VS Histograms

Both histograms and summaries track distributions of values like request durations or response sizes. They both expose _sum and _count, letting you calculate averages. The key concept they share is quantiles — e.g. the 0.95 quantile (95th percentile) means "95% of requests completed within X ms."

Histograms are the sensible default.
They expose raw bucket counts and compute quantiles on the server via histogram_quantile(). Because they store raw counts, you can sum them across multiple instances and compute a true quantile over all of them. The tradeoff is that quantiles are estimated via linear interpolation between buckets, so accuracy depends on how well your bucket boundaries match your data.

Summaries exist for one niche reason: exact quantile accuracy.
They compute quantiles directly on the client, so the result is always precise regardless of data distribution.

For example, if you have a single payment processing service and need to guarantee that 99% of transactions complete under 500ms for regulatory reporting, a summary gives you that exact number.

The cost is high though: quantiles are pre-computed and fixed (you can't query new ones later), they're more expensive on the client, and they can't be aggregated across instances since averaging pre-computed quantiles is mathematically incorrect.

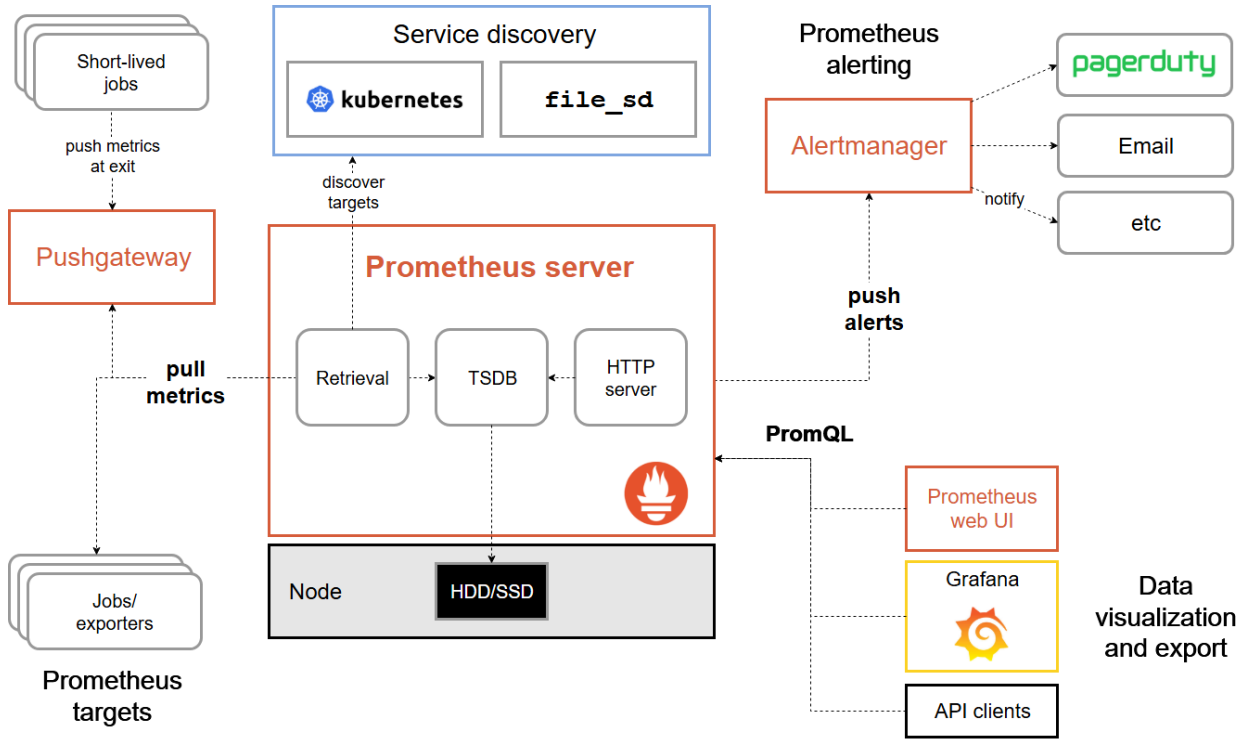
Architecture

Components

The Prometheus ecosystem consists of multiple components, many of which are optional:

- the main Prometheus server which scrapes and stores time series data.
- client libraries for instrumenting application code.
- a push gateway for supporting short-lived jobs.
- special-purpose exporters for services like HAProxy, StatsD, Graphite, etc..
- an alertmanager to handle alerts.
- various support tools.

Architecture



When Prometheus starts, it uses **service discovery** (or static config) to find its targets

— these can be Kubernetes pods, EC2 instances, exporters, or any HTTP endpoint exposing metrics.

Every `scrape_interval`, the Prometheus server pulls metrics from each target's **/metrics** endpoint and stores them in its local **TSDB** (time series database) on disk.

Short-lived jobs that can't be scraped directly push their metrics to the **Pushgateway** instead, which Prometheus then scrapes like any other target.

When alerting rules evaluate to true, Prometheus pushes alerts to the **Alertmanager**, which handles deduplication, grouping, silencing, and routing notifications to the right channel (email, PagerDuty, Slack, etc.).

The stored data can then be queried via **PromQL** — either through the built-in web UI, Grafana, or directly via the HTTP API.

Prometheus server

The Prometheus server is a single binary called Prometheus.
Example config:

```
global:                         # Server-wide default settings
  scrape_interval:     15s      # Collect metrics from targets every 15 seconds
  evaluation_interval: 15s      # Re-evaluate alerting rules every 15 seconds

rule_files:                     # List of files containing alerting/recording rules
  # - "first.rules"             # (commented out) would load rules from first.rules
  # - "second.rules"            # (commented out) would load rules from second.rules

scrape_configs:                 # Defines what to monitor
  - job_name: prometheus        # Name this monitoring job "prometheus"
    static_configs:             # Use a hardcoded list of targets (no auto-discovery)
      - targets: ['localhost:9090']  # Scrape metrics from itself at this address
```

Jobs and Instances

An instance is a single scrape endpoint (usually one process).
A job is a group of instances serving the same purpose.

Example:

```
- job: `api-server`
  - instance 1: `1.2.3.4:5670`
  - instance 2: `1.2.3.4:5671`
  - instance 3: `5.6.7.8:5670`
  - instance 4: `5.6.7.8:5671`
```

Prometheus automatically attaches `job` and `instance` labels to every scraped time series. It also stores these metrics per scrape automatically:

- `up` — `1` if reachable, `0` if not. Useful for availability monitoring.
- `scrape_duration_seconds` — how long the scrape took.

- `scrape_samples_scraped` — how many samples the target exposed.
- `scrape_samples_post_metric_relabeling` — how many samples remained after relabeling.
- `scrape_series_added` — approximate number of new series in this scrape. *(v2.10+)*

With the `extra-scrape-metrics` feature flag, you also get:
- `scrape_timeout_seconds` — the configured scrape timeout for the target.
- `scrape_sample_limit` — the configured sample limit (`0` if no limit).
- `scrape_body_size_bytes` — uncompressed size of the last scrape response. Returns `-1` if `body_size_limit` was exceeded, `0` on other failures.

Exporters

An exporter is a small server that sits beside a third-party system (like MySQL, Redis, or a Linux host), reads its internal metrics, and converts them into the Prometheus format so Prometheus can scrape them.
You use an exporter when you can't directly instrument the system with a Prometheus client library.
Some software (like Kubernetes, Grafana, or Etcd) already exposes metrics in Prometheus format natively — no exporter needed.

Official exporters (maintained by the Prometheus team) include:
- `node_exporter` — Linux/macOS host metrics (CPU, memory, disk, network)
- `blackbox_exporter` — probes endpoints over HTTP, HTTPS, DNS, TCP, ICMP
- `mysql_exporter` — MySQL server metrics
- `haproxy_exporter` — HAProxy stats
- `memcached_exporter` — Memcached stats
- `snmp_exporter` — SNMP devices
- `statsd_exporter` — StatsD metrics bridge
- `cloudwatch_exporter` — AWS CloudWatch metrics

A full list of community exporters is available in the Prometheus docs.

Key rules when writing your own exporter:
- Use base units (seconds, bytes) — never milliseconds or kilobytes
- Name metrics with a prefix matching the exporter, e.g. haproxy_up
- Only scrape when Prometheus pulls — don't run your own internal timer
- Deploy one exporter per instance of the monitored system, ideally on the same machine
- Don't expose machine-level metrics (CPU, memory) — that's the node exporter's job
- Expose an up metric (0 or 1) so Prometheus can track whether the scrape succeeded

Pushgateway

The Pushgateway is an intermediary service for cases where Prometheus can't scrape a target directly — mainly **short-lived batch jobs** that finish before Prometheus would ever scrape them.
The job pushes its metrics to the Pushgateway, and Prometheus scrapes the

Pushgateway instead.

When to use it

Only for service-level batch jobs — jobs that aren't tied to a specific machine or instance. For example, a nightly job that purges deleted user accounts. These jobs start, do their work, and exit before Prometheus can scrape them.

When NOT to use it

For almost everything else. The Pushgateway has significant drawbacks:

- It becomes a single point of failure if multiple jobs push through it.
- You lose the automatic up health monitoring that comes with normal scraping.
- It never forgets metrics — if a job pushes metrics and then disappears, those metrics stay in the Pushgateway forever until you manually delete them.

Alternatives to consider

- If a firewall is blocking scrapes, move Prometheus inside the network or use PushProx.
- For machine-level batch jobs (cron jobs, config management runs), use the Node Exporter's textfile collector instead.

Alerting

Alerting in Prometheus is split into two parts: Prometheus fires alerts based on rules, and the **Alertmanager** handles what happens with those alerts — deduplication, grouping, routing, silencing, and sending notifications (email, PagerDuty, Slack, etc.).

Setup steps

1. Install and configure Alertmanager
2. Point Prometheus at your Alertmanager
3. Write alerting rules in Prometheus

Core Alertmanager concepts

- **Grouping** — Bundles similar alerts into one notification. For example, if 200 instances lose database connectivity at once, you get one grouped alert instead of 200 individual ones. Configured via a routing tree.
- **Inhibition** — Suppresses lower-level alerts when a higher-level one is already firing. For example, if an entire cluster is down, mute all the individual service alerts for that cluster since they're just noise.
- **Silences** — Manually mute alerts for a set time window. Configured in the Alertmanager web UI.

Alerting Rules

Alerting rules define conditions that, when met, trigger an alert. They live in rule files alongside recording rules.

```yaml
groups:
  - name: example
    rules:
        # Alert name
      - alert: HighRequestLatency
        # Condition to trigger the alert
        expr: job:request_latency_seconds:mean5m{job="myjob"} > 0.5
        # Must be true for 10m before firing
        for: 10m
        # Keep firing 5m after condition clears
        keep_firing_for: 5m
        # Extra labels attached to the alert
        labels:
          # Severity level, e.g., page, ticket, warning, etc.
          severity: page
        # Human-readable info (description, runbook, etc.)
        annotations:
          # A human-readable description of the alert
```

Key fields

- **`expr`** — the PromQL condition that triggers the alert
- **`for`** — how long the condition must hold before the alert fires (prevents flapping). During this window the alert is in **pending** state.
- **`keep_firing_for`** — how long to keep the alert firing after the condition clears (prevents false resolutions)
- **`labels`** — extra labels attached to the alert (e.g. severity)
- **`annotations`** — human-readable info like descriptions or runbook links. Supports templating with **`{{ $labels.instance }}`** and **`{{ $value }}`**

Alert states

`pending` → **`firing`** → resolved
Alerting rules only detect the problem. Routing, deduplication, silencing and actual notifications are handled by the **Alertmanager**.

Recording Rules

Pre-compute an expensive or repeated PromQL expression and store the result as a new metric. Prometheus evaluates it every evaluation_interval and stores it like any other time series.

```yaml
groups:
  - name: example
    rules:
      - record: job:request_latency_seconds:mean5m  # new metric name
        expr: rate(request_latency_seconds_sum[5m]) # expression to pre-compute
              /
              rate(request_latency_seconds_count[5m])
```

The result is then queryable like any normal metric:
`job:request_latency_seconds:mean5m{job="myjob"}`

High Availability

You can run multiple Alertmanager instances as a cluster. The key design goal is **fail open** — during outages or network splits, Alertmanager prefers sending duplicate notifications over missing alerts.

How it works

Each instance receives alerts independently from Prometheus (Prometheus sends to all instances, never through a load balancer). Instances communicate via a **gossip protocol** (Hashicorp Memberlist) to share three things: silences, notification logs, and membership state.

Deduplication via staggered waiting

To avoid all instances sending the same notification, each instance waits based on its position in the sorted peer list:

- Instance 0 (AM-1): waits 0s → sends notification → logs it
- Instance 1 (AM-2): waits 15s → sees AM-1's log entry → skips
- Instance 2 (AM-3): waits 30s → sees AM-1's log entry → skips

If AM-1 fails, AM-2 becomes position 0 and takes over. The cluster tolerates up to N-1 failures.

Network partition (split-brain)

If the cluster splits, each partition sends its own notification independently. This results in duplicate notifications, which is intentional — missing an alert is worse than a duplicate.

Silence replication

Silences created on any instance are gossiped to all peers. All instances converge to the same silence state, so you can manage silences from any instance.

Key operational rules

- Always configure Prometheus to send to all Alertmanager instances directly — never use a load balancer in between
- Cluster communication runs on port 9094 (TCP + UDP) — make sure it's open between instances
- In cloud/NAT environments, explicitly set --cluster.advertise-address
- Odd vs. even cluster sizes don't matter — there's no quorum or voting
- 2-3 instances is typical for production; 4-5 for multi-datacenter setups

Prometheus Global Config

Global config

Setting	Default	Description
scrape_interval	1m	How often to collect metrics
scrape_timeout	10s	How long before a scrape fails (must be ≤ scrape_interval)
evaluation_interval	1m	How often rules are evaluated
external_labels	—	Labels added to all metrics sent to external systems
body_size_limit	0 (no limit)	Max response size before a scrape fails
sample_limit	0 (no limit)	Max samples accepted per scrape
query_log_file	—	File to log all PromQL queries to

Scrape config

Setting	Default	Description
job_name	required	Unique name for the job
scrape_interval	global	Overrides global scrape interval for this job
scrape_timeout	global	Overrides global scrape timeout for this job
metrics_path	/metrics	Path to scrape metrics from
scheme	http	http or https
static_configs	—	Hardcoded list of targets
honor_labels	false	Keep scraped labels on conflict instead of renaming them
honor_timestamps	true	Use timestamps from scraped data
relabel_configs	—	Transform target labels before scraping
metric_relabel_configs	—	Transform or drop samples after scraping
sample_limit	0 (no limit)	Max samples per scrape for this job
body_size_limit	0 (no limit)	Max response size for this job

Relabel config actions

Action	Description
replace	Match a label with regex and rewrite its value
keep	Only keep targets where regex matches
drop	Drop targets where regex matches
labelmap	Copy labels matching regex to new names
labeldrop	Remove all labels matching regex
labelkeep	Remove all labels NOT matching regex
lowercase / uppercase	Convert label value to lower/upper case

Remote write

Setting	Default	Description
url	required	Remote endpoint to forward metrics to
remote_timeout	30s	Timeout per request
write_relabel_configs	—	Filter which metrics are forwarded
send_native_histograms	false	Whether to forward native histograms
queue_config.capacity	10000	Samples buffered per shard
queue_config.max_shards	50	Max concurrency
queue_config.max_samples_per_send	2000	Max samples per batch

Remote read

Setting	Default	Description
url	required	Remote endpoint to query from
remote_timeout	1m	Timeout per request
read_recent	false	Query remote even when local data is sufficient
required_matchers	—	Only use this remote for queries with specific labels
filter_external_labels	true	Use external labels as selectors when querying

TSDB (nice to know)

Setting	Default	Description
out_of_order_time_window	0s (disabled)	How far out-of-order samples are accepted
retention.time	15d	How long to keep data
retention.size	0 (no limit)	Max disk size for blocks

PromQL

PromQL is Prometheus's query language. You use it to select, filter, and aggregate time series data.
Queries can be **instant** (a single point in time) or **range** (over a time window). In the Prometheus UI, the "Table" tab is for instant queries and the "Graph" tab is for range queries.

Selectors

Select all time series with a metric name:
`http_requests_total`
Filter by labels using `{}`:
`http_requests_total{job="apiserver", handler="/api/comments"}`

Operator	Meaning
=	exact match
!=	not equal
=~	regex match
!~	regex not match

```
http_requests_total{job=~".*server"}   # jobs ending with "server"
http_requests_total{status!~"4.."}     # exclude 4xx status codes
```

Select a range of data (last 5 minutes):

```
http_requests_total{job="apiserver"}[5m]
```

Key Functions

Function	Use case
rate(v[d])	Per-second average rate of increase over a window. Use with counters.
increase(v[d])	Total increase over a window. Syntactic sugar for rate() × seconds.
irate(v[d])	Per-second rate based on the last two data points. Use for volatile counters. Don't use for alerts.
histogram_quantile(φ, v)	Compute a quantile from a histogram.
absent(v)	Returns 1 if a metric has no data. Useful for alerting on missing metrics.
predict_linear(v[d], t)	Predicts a gauge's value t seconds from now using linear regression.
delta(v[d])	Difference between first and last value in range. Use with gauges only.
avg_over_time(v[d])	Average value of a series over a time window.

```
rate(http_requests_total[5m])
histogram_quantile(0.9, rate(http_request_duration_seconds_bucket[10m]))
predict_linear(node_filesystem_free_bytes[1h], 4*3600)  # will disk be
full in 4h?
```

Aggregation Operators

Operator	Description
sum	Sum all values
avg	Average of all values
min / max	Minimum or maximum value
count	Count number of series
topk(k, v)	Top k series by value
bottomk(k, v)	Bottom k series by value
quantile(φ, v)	φ-quantile across series

Use by to keep a label dimension, or without to drop one:

```
# sum, keeping job label
sum by (job) (rate(http_requests_total[5m]))
```

```
# sum, dropping instance label
sum without (instance) (rate(http_requests_total[5m]))

# top 3 apps by request rate
topk(3, sum by (app) (rate(http_requests_total[5m])))

# number of instances per app
count by (app) (http_requests_total)
```

Arithmetic & Comparisons

```
# Free memory in MiB per instance
(instance_memory_limit_bytes - instance_memory_usage_bytes) / 1024 /
1024

# Error rate ratio
rate(http_errors_total[5m]) / rate(http_requests_total[5m])
```

Comparison operators filter by default — series where the condition is false are dropped. Add bool to return 0/1 instead:

Mode	Behavior
http_requests_total > 100	Only returns series where value > 100
http_requests_total > bool 100	Returns 1 where true, 0 where false

Practical Tips

- Always apply rate() before aggregating — otherwise counter resets won't be detected correctly.
- Start queries in Table view before switching to Graph — a bare metric name can expand to thousands of series.
- If a query is too slow to run ad-hoc, pre-compute it with a **recording rule**.
- Use offset to compare against the past: `rate(http_requests_total[5m] offset 1w)` gives last week's rate.